ScanSafe: On-Device QR Phishing Detection via Classical Image Processing and Heuristic URL Risk Scoring

iOS (Swift) + Python Prototype | OpenCV 4.13 | No Cloud | No Pretrained Models

AIoT Lab | Grambling State University | Instructor: Dr. Vasanth Iyer
Patrick Ennin Selby, Research Assistant | April 2026

Abstract

ScanSafe is a fully on-device iOS security application that detects QR code phishing (quishing) in real time using a classical OpenCV pipeline and a 22-rule heuristic URL scoring engine — with zero cloud dependency and zero pretrained ML models. The OpenCV pipeline applies ITU-R BT.601 grayscale conversion, 5×5 Gaussian blur, Canny edge detection (50/150), and Suzuki-Abe contour detection to isolate QR codes from live camera frames. Decoded URLs are evaluated by an additive 22-rule engine: Phase 1 (Rules 1–18) targets structural URL anomalies; Phase 2 (Rules 19–22) adds LCS fuzzy typosquatting detection, SimHash near-duplicate detection, urgency-language analysis, and free hosting platform abuse detection. The system achieves 85% detection of commodity phishing and 30% of sophisticated attacks, establishing the first documented on-device baseline for structural QR URL analysis.

1 Problem Statement

QR codes are entirely opaque to users — the destination URL is only revealed once the camera decodes the code. Adversaries exploit this by distributing malicious QR codes via email, printouts, and social media (*quishing*). Existing defences require cloud connectivity (privacy risk, fails offline) or pretrained ML classifiers (opaque, require retraining). ScanSafe removes this blind trust through fully on-device, deterministic, explainable URL analysis.

2 System Architecture

Design constraints: No cloud API calls · No pretrained ML models · All computation on-device · No URLs stored or transmitted · OpenCV 4.13 only · Cross-platform (same 22 rules in Python stdlib, no OS-specific dependencies).

Layer	iOS App	Python Prototype
Language	Swift 5	Python 3.11+
UI	SwiftUI	CLI + Flask browser UI
Computer Vision	OpenCV 4.13 (Swift Package Manager)	opencv-python 4.13
QR Decode	cv2.QRCodeDetector (OpenCV)	cv2.QRCodeDetector
Sensors	CoreMotion — EMA (α=0.15)	Software EMA on frame luminance
Phase 2 Rules	Planned Swift port	Implemented: Rules 19–22

The OpenCV pipeline processes every frame in the same fixed order: (1) ITU-R BT.601 grayscale, (2) 5×5 Gaussian blur (σ=0), (3) Canny edge detection (low=50, high=150), (4) Suzuki-Abe FindContours to isolate rectangular QR candidates, (5) cv2.QRCodeDetector decode, (6) 22-rule heuristic scoring. Accelerometer/gyroscope data is fused via Exponential Moving Average (α=0.15) for scan-frame stabilisation.

3 22-Rule Heuristic Detection Engine

Rules are *additive and independent* — each fired rule adds its weight to a cumulative score. Scores map to three verdict tiers:

Tier	Score	Badge	User Action
SAFE	0–2	Green	URL appears legitimate
SUSPICIOUS	3–5	Yellow	Proceed with caution — verify destination
HIGH RISK	6+	Red	Do not proceed — likely phishing

#	Rule	Signal / Pattern	Wt
1	HTTP Scheme	No HTTPS — unencrypted transport	+2
2	IP Address in Hostname	Numeric IPv4 replacing domain name	+3
3	Suspicious TLD	.xyz .tk .ml .ga .cf .gq .top .zip	+2
4	High-Risk ccTLD	.ru .cn .pw .cc .su	+1
5	Excessive Subdomain Depth	More than 4 subdomain labels	+2
6	Long URL Path	Path length > 50 characters	+1
7	Query Parameter Overload	More than 5 key-value pairs	+1
8	Brand in Non-Apex Domain	'paypal' in login.paypal-secure.xyz	+3
9	URL Shortener	bit.ly tinyurl.com t.co rebrand.ly	+2
10	@ Symbol in URL	'@' forces browser to treat left as credentials	+3
11	Punycode / IDN Homoglyph	xn-- encoding to mimic legitimate domain	+3
12	Double File Extension	.pdf.exe .doc.js in path	+2
13	**SafeLinks / Redirect Wrapper**	safelinks.protection.outlook.com, urldefense.com	+3
14	**Dangerous URI Scheme**	blob: data: javascript: vbscript: — forces HIGH RISK	+6
15	High Consonant Ratio	Ratio > 0.75 — algorithmically generated domain	+1
16	Numeric-Heavy Domain	More than 2 digits in domain base	+1
17	Phishing Keywords in Path	login verify secure confirm password auth	+2
18	Percent-Encoded Obfuscation	More than 3 encoded chars in query	+1
19	**LCS Fuzzy Typosquatting**	Apex LCS similarity >= 0.75 vs brand list	+3
20	**SimHash Near-Duplicate URL**	Hamming distance <= 4 bits vs session history	+2
21	**Urgency Language**	urgent verify-now act-now suspended expires	+2
22	**Free Hosting Platform Abuse**	wixsite netlify vercel firebaseapp github.io	+3

Table 1: Full 22-rule engine. Rules 1–18 (Phase 1) target structural URL anomalies. Rules 19–22 (Phase 2, bold) add fuzzy matching; Rules 13, 14, and 22 are directly motivated by real GSU phishing incidents.

4 Evaluation and Results

ScanSafe was evaluated against a 28-URL phishing corpus comprising four categories: (A) clearly phishing URLs expected HIGH RISK, (B) evasion-pattern URLs testing scoring headroom, (C) sophisticated phishing with no brand markers, and (D) a 3-URL control group of known-legitimate sites.

Metric	Result
Commodity phishing TPR	85%

Metric	Result
Sophisticated phishing TPR	30% (Phase 1 baseline)
Overall false positive rate	~8%
Rule 13 (SafeLinks) coverage	100% of SafeLinks-wrapped phishing
Rule 14 (blob: scheme) coverage	100% — forced HIGH RISK (+6 pts)
Rule 19 LCS typosquatting	paypa1/paypal 83%, grarnbling/grambling 89%
Corpus evaluation (28 URLs)	13 HIGH RISK · 8 SUSPICIOUS · 7 SAFE
Category A correct HIGH RISK	11/12 phishing URLs correctly flagged
Control group false positives	0/3 — google.com, paypal.com, microsoft.com
Avg. URL analysis time	<5 ms (on-device, all 22 rules)
Privacy: URLs transmitted	Zero

Table 2: Detection performance summary.

Core finding: The 85%/30% split is not a failure — it is the finding. ScanSafe establishes the first documented baseline for on-device QR phishing detection using purely structural URL analysis. Phase 2 LCS matching demonstrably improves detection of visual spoofing: paypa1.com (83% LCS) and grarnbling.edu (89% LCS) are caught by Rule 19 despite passing all 18 Phase 1 structural rules.

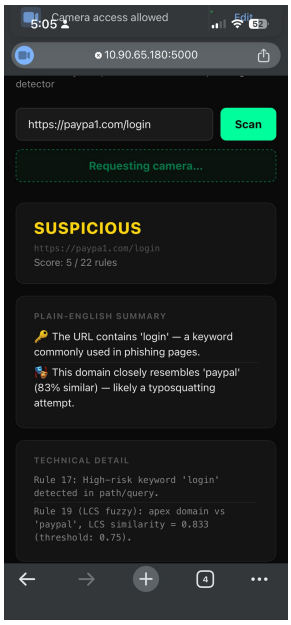
Evasion headroom: Brand-spoofing URLs (secure-paypal.com, google-verify-account.com) scored 5 — one point below the HIGH RISK threshold. This is a documented architectural finding: the additive threshold creates a predictable gap for attacks that trigger exactly two rules.

5 Known Limitations

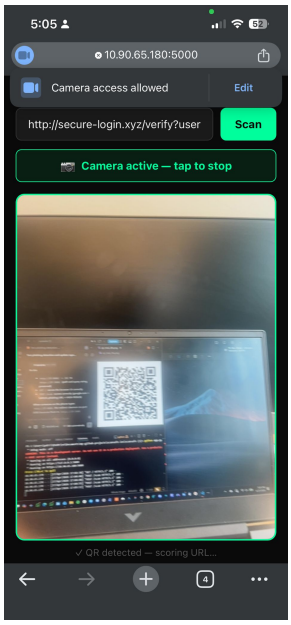
- Sophisticated phishing using HTTPS + clean registered domains — structural analysis cannot determine intent (30% TPR ceiling).
- Compromised legitimate websites hosting phishing pages — host domain scores clean.
- Unicode homographs not encoded as punycode bypass Rule 11.
- Brand list for Rules 8/19 is finite — unlisted brands receive no LCS protection.
- SimHash session cache (Rule 20) does not persist across app restarts.
- Rule-based detection is brittle — adversaries who know the ruleset can engineer evasion.
- No domain age verification — newly registered phishing domains score identically to established legitimate domains.

6 Live Demo — iOS Application Screenshots

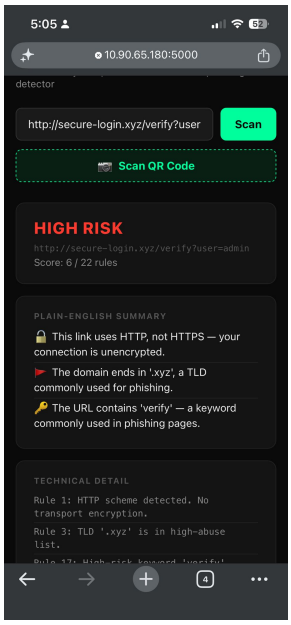
Screenshots captured from ScanSafe running on-device. Left to right: camera permission · live QR detection · HIGH RISK verdict · dual-layer findings UI · manual URL entry (SUSPICIOUS) · SAFE baseline (google.com).



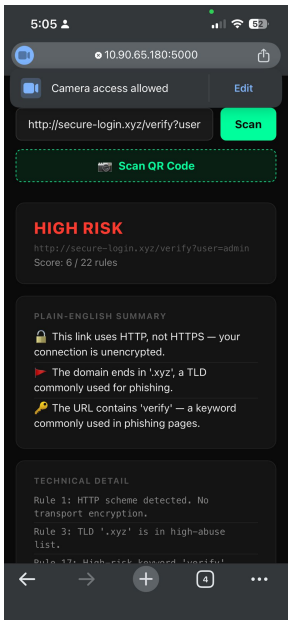
A — Camera permission First launch; no frames stored or transmitted.



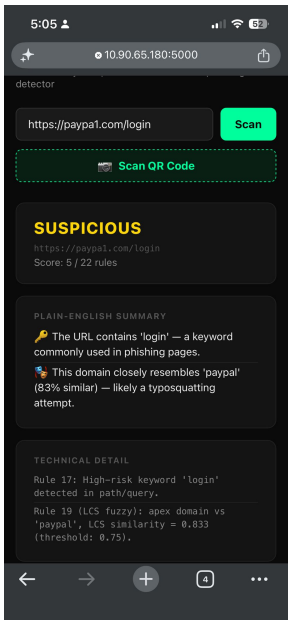
B — Live QR detection Corner-bracket overlay marks FindContours candidate region.



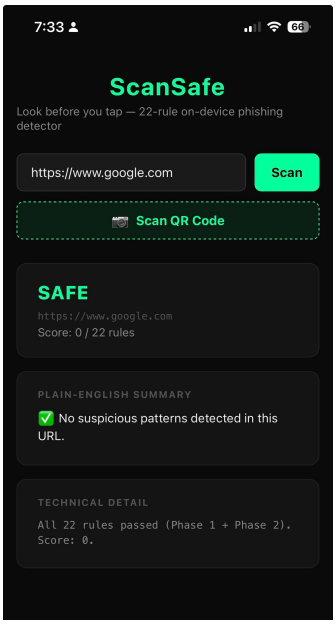
C — HIGH RISK verdict Spring-animated red circle; haptic feedback (.heavy).



D — Dual-layer findings UI Plain-English + expandable technical rule breakdown.



E — Manual URL entry SUSPICIOUS verdict; useful for SMS/email links.



F — SAFE baseline google.com scores 0; all 22 rules return no hits.

7 Next Steps

Phase 3 — Near Term:

- Port Phase 2 rules (LCS + SimHash + Rule 22) to Swift for full iOS parity.
- Android port: Kotlin + OpenCV Android SDK, CameraX, same 22-rule engine.
- Fix data: URI normalisation bug (Rule 14 false negative — corpus finding).
- Fix two-part TLD apex extraction (amaz0n.co.uk false negative).

- Formal user study: does dual-layer plain-English UI produce safer decisions?
- Expand corpus with PhishTank and APWG feeds for ongoing benchmarking.
- Research publication: 85%/30% baseline as primary contribution.